

## /prompt

Agent system-prompt inspector: `/prompt help|primary [agent]|subagent [agent]|list`

### Overview

`/prompt` renders the system prompt that a given agent would receive, without calling the LLM and without writing anything. It is a read-only inspector: every form builds the report from live session context (model, tools, project guidelines) and prints it. Two modes exist: the primary (interactive) prompt, and the subagent prompt, which forces Subagent mode - gating the mandatory Sub-Agent Completion Protocol section and excluding interactive tools from the tool surface.

### Syntax

```
/prompt
/prompt help
/prompt list
/prompt primary [agent]
/prompt subagent [agent]
/prompt <agent>
```

Notes:

- The bare form is equivalent to `/prompt help`.
- Agent names resolve case-insensitively; a bare `<agent>` form produces a primary-mode report.

### Options / Subcommands

| Form                                                 | Description                                                                                                          |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>/prompt</code> (or <code>/prompt help</code> ) | Print the help page (status: <code>prompt: help</code> ).                                                            |
| <code>/prompt list</code>                            | Print the roster of built-in and custom agents with the agent count (status: <code>prompt: list (N agents)</code> ). |
| <code>/prompt primary [agent]</code>                 | Render the primary-mode system-prompt report for the session agent or the named agent.                               |
| <code>/prompt subagent [agent]</code>                | Render the subagent-mode report; Subagent mode is forced regardless of the session mode.                             |
| <code>/prompt &lt;agent&gt;</code>                   | Case-insensitive agent-name shortcut for a primary-mode report.                                                      |
| unknown subcommand                                   | Print a usage correction with no prompt content (status: <code>prompt: unknown subcommand{req}"</code> ).            |

### Examples

Inspect the prompt the coder agent would receive right now:

```
/prompt
/prompt primary coder
```

See what a spawned sub-agent actually gets (completion protocol on, interactive tools removed):

`/prompt subagent coder`

Check the roster before picking an agent:

`/prompt list`

Short form, case-insensitive:

`/prompt CODE-REVIEW`

Unknown subcommand prints a correction instead of a prompt:

`/prompt foo bar`

## Output

- Report modes: `primary` renders the interactive-agent prompt; `subagent` renders the Subagent-mode prompt. Subagent mode gates the Sub-Agent Completion Protocol (MANDATORY - HARD REQUIREMENT) section and excludes interactive tools (for example `ask_user`) from the tool reference.
- The report is the rendered system prompt: agent instructions, project guidelines (`AGENTS.md`), tool reference, and mode-specific sections, all resolved against the live session (selected model, registered tools, working directory) rather than stored defaults.
- Status line on a report: `prompt: {agent} ({primary|subagent}, {N} tools)`; an agent with no tools shows `( ..., no tools)`.
- `list`: the built-in and custom agent roster plus the agent count.
- `help`: the help page (bare form included).
- Unknown subcommand: a usage correction only - no prompt body is printed.

Read-only guarantee: no LLM call is made, no config or session state is modified, and nothing is persisted.

## Related

- Agent presets and custom agents (OASF JSON / Markdown profiles): `docs/custom-agents.md`.
- `/agents` to list loaded agents and diagnostics, `/agent` for the picker.
- `/system` for a session-scoped system-prompt override.
- Sub-agent spawning: `new_agent`, `wait_agents`, `agent_complete`.